

W05 — pgsodium Envelope Encryption for Provider API Keys

Goal: Replace the current base64 "encryption" of BYO-LLM provider API keys with real envelope encryption — a per-project Data Encryption Key (DEK) wrapped by a Key-Encryption-Key (KEK) held in the Postgres key ring (pgsodium), AES-256-GCM at the row level, plaintext never leaving the database.

Satisfies: PRD NF-032 (at-rest encryption of secrets), NF-033 (KMS-wrapped DEK), NF-U033 (revoked keys rejected ≤10s), NF-035 (24h rotation grace), US-A039 (admin-only key read), US-A042 (rotation SLA: new key active ≤60s).

Status: Plan only. No source edits in this file.

0. Feasibility: pgsodium on Supabase

Plan	pgsodium	Vault	Notes
Supabase Free	DEPRECATED	Yes	pgsodium was removed from hosted Supabase on 2024-04; Vault remains.
Supabase Pro+	Same	Yes	Same.
Self-hosted PG 15+	Available	n/a	Extension can be installed manually.

Decision: Because the project targets hosted Supabase, we **use supabase_vault as the primary path** (`vault.create_secret / vault.decrypted_secrets` view) and **keep pgsodium as an opt-in fallback** for self-hosted. The terminology "pgsodium" in the migration filename is preserved for PRD trace, but the implementation is `vault`-first. If neither is available, we fall back to app-layer AES-256-GCM with the KEK in `APP_KEK_B64` env (32-byte key), flagged as a follow-up KMS migration (AWS KMS / GCP KMS / Azure Key Vault).

The `secret-store.ts` service abstracts the three backends behind one interface — callers never know which is active.

1. Migration — supabase/migrations/20260508_0001_pgsodium_providers.sql

1.1 Extension bootstrap (idempotent, branch by availability)

```
-- Prefer supabase_vault (hosted Supabase). Fall back to pgsodium (self-hosted).
DO $$
BEGIN
  IF EXISTS (SELECT 1 FROM pg_available_extensions WHERE name = 'supabase_vault') THEN
    CREATE EXTENSION IF NOT EXISTS supabase_vault WITH SCHEMA vault;
  ELSIF EXISTS (SELECT 1 FROM pg_available_extensions WHERE name = 'pgsodium') THEN
    CREATE EXTENSION IF NOT EXISTS pgsodium;
  ELSE
    RAISE NOTICE 'Neither supabase_vault nor pgsodium available – app-layer AES-GCM fallback will be used.';
  END IF;
END $$;
```

1.2 Schema changes on providers

```

ALTER TABLE public.providers
  ADD COLUMN IF NOT EXISTS vault_secret_id UUID,          -- vault path (primary)
  ADD COLUMN IF NOT EXISTS dek_id          UUID,          -- pgsodium key ring id (fallback)
  ADD COLUMN IF NOT EXISTS nonce          BYTEA,          -- AES-GCM nonce (fallback + app-layer)
  ADD COLUMN IF NOT EXISTS ciphertext      BYTEA,          -- AES-GCM ciphertext (fallback + app-layer)
  ADD COLUMN IF NOT EXISTS key_version     INTEGER NOT NULL DEFAULT 1,
  ADD COLUMN IF NOT EXISTS previous_vault_secret_id UUID, -- 24h rotation grace (NF-035)
  ADD COLUMN IF NOT EXISTS rotated_at      TIMESTAMPTZ,
  ADD COLUMN IF NOT EXISTS revoked_at      TIMESTAMPTZ;

-- status already allows 'revoked' via routes/providers.ts; widen the check:
ALTER TABLE public.providers DROP CONSTRAINT IF EXISTS providers_status_check;
ALTER TABLE public.providers ADD CONSTRAINT providers_status_check
  CHECK (status IN ('active','suspended','error','revoked','rotated'));

CREATE INDEX IF NOT EXISTS idx_providers_status ON public.providers(status);
CREATE INDEX IF NOT EXISTS idx_providers_vault_secret ON public.providers(vault_secret_id);

```

1.3 Decryption helper + ADMIN-gated view

```

-- Returns plaintext for a given provider or NULL. SECURITY DEFINER so the
-- view can traverse vault.decrypted_secrets without granting the caller
-- direct access. RLS below restricts WHO may SELECT from the view.
CREATE OR REPLACE FUNCTION public.fn_provider_plaintext(p_provider_id UUID)
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, vault
AS $$
DECLARE
  v_secret_id UUID;
  v_plain      TEXT;
BEGIN
  SELECT vault_secret_id INTO v_secret_id
    FROM public.providers
   WHERE id = p_provider_id AND status IN ('active','rotated')
   LIMIT 1;

  IF v_secret_id IS NULL THEN RETURN NULL; END IF;

  SELECT decrypted_secret INTO v_plain
    FROM vault.decrypted_secrets
   WHERE id = v_secret_id;

  RETURN v_plain;
END $$;

REVOKE ALL ON FUNCTION public.fn_provider_plaintext(UUID) FROM PUBLIC;

CREATE OR REPLACE VIEW public.providers_decrypted AS
SELECT
  p.id,
  p.owner_id,
  p.name,
  p.provider_type,
  p.model_id,
  p.base_url,
  p.status,
  p.key_version,
  public.fn_provider_plaintext(p.id)                AS api_key_plaintext,
  public.fn_provider_plaintext_previous(p.id)        AS api_key_plaintext_previous,

```

```

    p.rotated_at,
    p.revoked_at
FROM public.providers p;

```

```

ALTER VIEW public.providers_decrypted SET (security_invoker = true);

```

```

-- RLS on the view (via underlying table): only ADMIN may SELECT.
-- The existing providers_select policy allows owner OR admin; tighten at
-- the view layer with a wrapper policy.

```

```

CREATE POLICY providers_decrypted_admin_only
ON public.providers
FOR SELECT
TO PUBLIC
USING (public.current_user_role() = 'ADMIN');

```

Note: a second helper `fn_provider_plaintext_previous` mirrors the above against `previous_vault_secret_id` for in-flight job decryption during the 24h rotation grace.

1.4 Backfill existing rows

```

-- Existing rows store base64(plaintext) in api_key_ciphertext. Migrate each
-- row into vault in a single DO block; wipe legacy columns afterward.

```

```

DO $$
DECLARE
    r RECORD;
    v_id UUID;
BEGIN
    FOR r IN
        SELECT id, api_key_ciphertext
        FROM public.providers
        WHERE vault_secret_id IS NULL
        AND api_key_ciphertext IS NOT NULL
    LOOP
        v_id := vault.create_secret(
            convert_from(decode(r.api_key_ciphertext, 'base64'), 'UTF8'),
            format('provider_api_key_%s', r.id),
            'BYO-LLM API key'
        );
        UPDATE public.providers SET vault_secret_id = v_id WHERE id = r.id;
    END LOOP;
END $$;

```

```

-- Defer the destructive drops to a follow-up migration (after backend
-- ships with the new reader) to allow rollback:
-- ALTER TABLE public.providers DROP COLUMN api_key_ciphertext;
-- ALTER TABLE public.providers DROP COLUMN api_key_nonce;

```

Fallback branch (pgsodium / app-layer): if vault is unavailable, the DO block instead uses `pgsodium.crypto_aead_det_encrypt(...)` keyed by a key ring entry allocated via `pgsodium.create_key(name => 'provider_api_keys_kek', key_type => 'aead-det')`, and stores (`dek_id`, `nonce`, `ciphertext`) on the row. The view swaps the inner SELECT accordingly.

2. backend/src/services/secret-store.ts (NEW)

Thin wrapper with three back-ends picked at startup:

```

// Interface
export interface SecretStore {
    put(providerId: string, plaintext: string): Promise<{ vaultSecretId: string }>;

```

```

    get(providerId: string, opts?: { allowPrevious?: boolean }): Promise<string>;
    rotate(providerId: string, newPlaintext: string): Promise<void>;
    revoke(providerId: string): Promise<void>;
  }

// Implementations, selected by env SECRETS_BACKEND = 'vault' | 'pgsodium' | 'app-aes'
class VaultSecretStore implements SecretStore { /* calls vault.create_secret / providers_decrypted */ }
class PgsodiumSecretStore implements SecretStore { /* calls pgsodium aead functions */ }
class AppAesSecretStore implements SecretStore { /* Node crypto.subtle AES-GCM, KEK from APP_KEK_B64 */ }

export function createSecretStore(sql, log): SecretStore { /* probe + pick */ }

```

Read path (all backends):

1. Check in-process LRU cache {providerId → {plaintext, expiresAt}} (TTL 8s — below the 10s revocation SLA).
2. On miss: `SELECT api_key_plaintext FROM providers_decrypted WHERE id = $1 AND status IN ('active', 'rotated')`.
3. On row miss or status = 'revoked': throw `ProviderRevokedError` (HTTP 409).
4. Write one row to `audit_log` with action='PROVIDER_KEY_READ', resource_type='provider', resource_id=providerId, actor_id=requesterId, meta={reason}. Never include the plaintext or last4 in meta.
5. Return plaintext. Never log it. All local variables holding plaintext are zero-width-scoped — use `try { use(key) } finally { /* no-op, GC handles; do NOT stringify err objects containing key */ }`.

Logging guard: add a pino redaction path for the top-level logger in `server.ts` — redact: `['req.body.apiKey', 'row.api_key_', 'apiKeyPlaintext', 'plaintext', '*.apiKey']`.

3. backend/src/services/llm-adapter.ts — patch resolveProvider

Replace lines ~358–388 so the function no longer reads `api_key_ciphertext` directly:

```

import { getSecretStore } from './secret-store.js';

export async function resolveProvider(
  providerId: string,
  requesterId: string,
  reason: 'llm_call' | 'health_check' | 'models_list',
): Promise<ProviderConfig> {
  const row = (await sql`
    SELECT id, provider_type, base_url, model_id, status
    FROM public.providers
    WHERE id = ${providerId}::uuid AND status IN ('active', 'rotated')
    LIMIT 1
  `[0]);
  if (!row) throw new Error(`Provider not found or inactive: ${providerId}`);

  const apiKeyPlaintext = await getSecretStore().get(providerId, { allowPrevious: row.status === 'rotated' });

  await auditKeyRead({ actorId: requesterId, providerId, reason });

  return {
    id: row.id,
    providerType: row.provider_type,
    baseUrl: row.base_url ?? undefined,
    modelId: row.model_id,
    apiKeyPlaintext,
  };
}

```

Call-site updates: every existing caller of `resolveProvider(id)` must now pass `(id, requesterUserId, reason)`. Diff the three callers:

- `backend/src/services/judge-runner.ts` (runs jobs) → pass `job.createdBy` and `'llm_call'`.
- `backend/src/routes/providers.ts` /health handler (line ~434) → replace the manual `decryptApiKey(...)` with `await getSecretStore().get(id)`, plus audit reason `'health_check'`.
- `backend/src/routes/providers.ts` /models handler (line ~907) → same with `'models_list'`.

Also delete the old helpers in `backend/src/services/crypto.ts` (`encryptApiKey`, `decryptApiKey`) and re-export shims that throw in dev and no-op in prod for one release, then remove entirely.

4. Rotation flow (POST /v1/providers/:id/rotate)

Patch to implement the 24h grace window so in-flight jobs decrypt the OLD key while new callers pick up the NEW key within ≤60s (NF-U033 / US-A042):

```
await sql.begin(async (tx) => {
  const [cur] = await tx`SELECT vault_secret_id, key_version FROM providers WHERE id = ${id} FOR UPDATE`;
  const newSecretId = await getSecretStore().putTx(tx, id, body.apiKey);
  await tx`
    UPDATE providers SET
      previous_vault_secret_id = ${cur.vault_secret_id},
      vault_secret_id         = ${newSecretId},
      key_version              = ${cur.key_version + 1},
      api_key_last4            = ${last4},
      rotated_at               = NOW(),
      status                   = 'active',
      updated_at               = NOW()
    WHERE id = ${id}
  `;
});
// Fire pg_notify so every backend node purges its LRU within ≤1s.
await sql`SELECT pg_notify('provider_key_changed', ${id}::text)`;
// Schedule a cleanup: after 24h (NF-035), null out previous_vault_secret_id
// and delete the old vault entry. Done by a periodic sweeper, not inline.
await auditLog({ action: 'KEY_ROTATED', ... meta: { key_version: cur.key_version + 1 } });
```

In-flight job reads (the grace):

- `secret-store.get(id, { allowPrevious: true })` falls back to `api_key_plaintext_previous` only when called inside a job whose `job.started_at < providers.rotated_at`.
- Judge runner passes `allowPrevious: job.startedAt < provider.rotatedAt && now < provider.rotatedAt + 24h`. After 24h window, reads fail loudly → job is retried against the new key or marked failed.

Sweeper: a new periodic task in `backend/src/jobs/rotate-sweeper.ts` (runs every 15 min) executes:

```
UPDATE providers SET previous_vault_secret_id = NULL
WHERE rotated_at < NOW() - INTERVAL '24 hours'
AND previous_vault_secret_id IS NOT NULL
RETURNING previous_vault_secret_id;
-- then: vault.update_secret(old_id, '{"archived": true}') or vault.delete_secret(old_id)
```

5. Revocation flow (POST /v1/providers/:id/revoke)

Augment the existing handler:

1. Transaction: set `status='revoked'`, `revoked_at=NOW()`.

2. Optionally call `vault.update_secret(vault_secret_id, 'REVOKED')` to guarantee any future plaintext read returns the literal "REVOKED" string (defense-in-depth — even if cache is stale, the adapter's upstream call 401s immediately).
3. `SELECT pg_notify('provider_key_changed', id) →` every backend's LRU listener evicts within $\leq 1s$.
4. LRU entries carry `expiresAt = inserted + 8s`, so in the worst-case (node missed the notify) the key is gone $\leq 8s$ later. Combined cap: $\leq 10s$ (NF-U033 ✓).
5. Audit action='KEY_REVOKED'.

LRU with pg_notify: at server boot, spin one dedicated postgres listener connection:

```
const listener = postgres(DATABASE_URL, { max: 1 });
await listener`LISTEN provider_key_changed`;
listener.notify = (msg) => secretLru.delete(msg.payload);
```

6. Tests — backend/test/secret-store.test.ts (NEW, vitest)

Minimum cases:

1. **Round-trip (all backends):** `put('abc123') → get() === 'abc123'`, parametrised over `['vault', 'pgsodium', 'app-aes']` — tests that don't find the extension skip cleanly via `test.skipIf(!hasVault)`.
2. **Rotation grace:** put key A; rotate to B; `get(id, { allowPrevious: true })` returns A; `get(id)` returns B. Advance mocked clock 24h → `{ allowPrevious: true }` now returns B (previous wiped).
3. **Revocation invalidation:** put key; revoke; `get()` throws `ProviderRevokedError`. Seed LRU manually with stale entry; issue `NOTIFY provider_key_changed, id`; assert LRU cleared within 2s.
4. **Audit emission:** every successful get appends exactly one `audit_log(kind='PROVIDER_KEY_READ')` row with `resource_id=providerId` and no plaintext in meta.
5. **No plaintext in logs:** spy on pino's transport; call `get()` then throw inside the consumer; assert the captured log lines never contain the plaintext string (byte-level search).
6. **Backfill idempotency:** run the migration DO block twice against the same test DB; row count in `vault.secrets` is stable.
7. **RLS:** a PG session running as role anon/annotator returns 0 rows from `providers_decrypted`; admin returns N rows.

Harness: uses the embedded-postgres fixture already wired in `backend/test/_setup.ts`; loads `supabase_vault` if the bundled PG has it (docker-postgres + `supabase/postgres:15` image), else runs only the `app-aes` variants.

7. Follow-ups (out of scope for W05)

- **KMS migration:** wrap `APP_KEK_B64` with AWS KMS Decrypt at boot. Needed if `app-aes` ever ships to prod.
 - **Drop legacy columns:** a follow-up migration `20260509_xxxx_drop_legacy_api_key_columns.sql` removes `api_key_ciphertext` + `api_key_nonce` after one full release cycle.
 - **Audit viewer:** surface `PROVIDER_KEY_READ` entries in `AdminAuditLog.tsx` with a per-provider filter (US-A039 traceability).
 - **Rate limiting:** cap `get()` per provider per minute to blunt a compromised-backend scenario; hard deny if > 500 reads/min.
 - **Key-ring scope:** the current plan uses a single project KEK. PRD hints at per-tenant KEKs — defer until multi-tenant is live.
-

8. File manifest

New files: - `supabase/migrations/20260508_0001_pgsodium_providers.sql` - `backend/src/services/secret-store.ts` - `backend/src/jobs/rotate-sweeper.ts` - `backend/test/secret-store.test.ts`

Edited files: - `backend/src/services/llm-adapter.ts` (resolveProvider signature + body) - `backend/src/services/crypto.ts` (deprecate `encryptApiKey` / `decryptApiKey`) - `backend/src/routes/providers.ts` (create/rotate/revoke/health/models call-sites) - `backend/src/services/judge-runner.ts` (pass requester + reason) - `backend/src/server.ts` (register `pg_notify` listener; pino redact list)

Env additions: - SECRETS_BACKEND = vault | pgsodium | app-aes (default: probe) - APP_KEK_B64 — required only when SECRETS_BACKEND=app-aes (32 raw bytes, base64)

9. Rollout order

1. Ship migration `20260508_0001_pgsodium_providers.sql` (additive only — no drops).
 2. Ship backend with `secret-store.ts` dual-reading: prefer vault, fallback to legacy base64 if `vault_secret_id IS NULL`. This keeps rollback safe.
 3. Run backfill (embedded in migration for hosted; manual one-shot for prod).
 4. Verify `SELECT COUNT(*) FROM providers WHERE vault_secret_id IS NULL = 0`.
 5. Flip reader to vault-only in the next release.
 6. Ship the drop-legacy-columns migration.
-

10. Acceptance checklist (maps to PRD)

- [] NF-032: no plaintext at rest — `api_key_ciphertext` column dropped by step 6; vault storage verified.
- [] NF-033: DEK wrapped by KEK — vault uses pgsodium internally; confirmed via `vault.secrets` having non-null `key_id`.
- [] NF-U033: revoked key rejected $\leq 10s$ — integration test asserts $\leq 8s$ LRU TTL + `pg_notify` path.
- [] NF-035: 24h rotation grace — sweeper test confirms cleanup at 24h.
- [] US-A039: every decrypt audited — vitest asserts 1:1 audit row per `get()`.
- [] US-A042: new key active $\leq 60s$ — rotation updates `vault_secret_id` in same TX, new reads pick it up on next call (well under 60s).